Upskilling Week
Barcelona

Mobile
WorldCapital
Barcelona

15:00h - 20:00h
Bravo Robots

RL en MuJoCo: desde el entrenamiento hasta el despliegue de políticas para el MicroDuck

Impulsat per

Mobile
WorldCapital
Barcelona

Amb la col·laboració de

Barcelona | Barcelona Activa | Generalitat de Catalunya

Powered by

TALENT ARENA

WORKSHOP · 16 NOV 2026 · 5 H

IA aplicada a un entorno físico

Cómo un robot aprende a andar sin que nadie le escriba las reglas.



Robot Microduck · simulación en MuJoCo

LO PRIMERO DE TODO · 8 MIN

Prepara tu PC ahora

1

Carpeta

Crea una carpeta vacía y ábrela en VS Code.

2

Clonar

En la terminal de VS Code, copia el proyecto con git clone.

3

Instalar

Doble clic en INSTALAR.bat. Unos 3 minutos, sin internet.

4

Seguir

Sigue sola mientras empieza la Ignición.

git clone https://github.com/david2526IA/workshop-ia-fisica .

Sabrás que ha terminado cuando la ventana ponga TODO LISTO.

¿Cómo ha aprendido a patinar?



Robot Microduck real · vídeo: Pollen Robotics

¿Cómo lo ha aprendido?

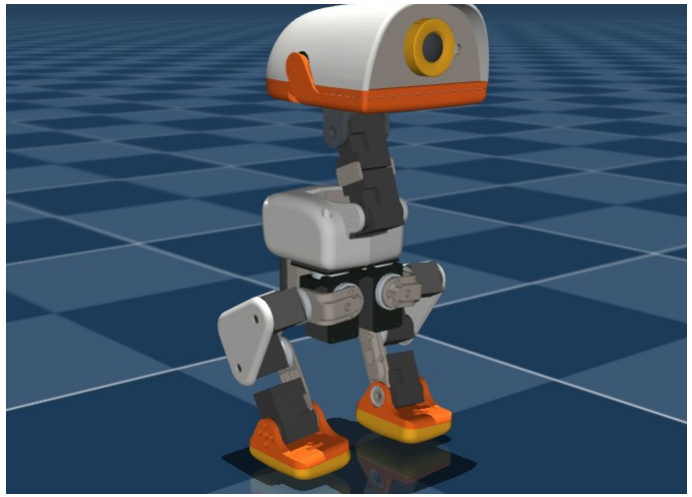
1. En simulación. Un robot virtual con ruedas bajo los pies.

2. Por ensayo y error. Miles de intentos a la vez en una GPU.

3. Con una recompensa. Puntos por seguir la velocidad pedida sin caerse.

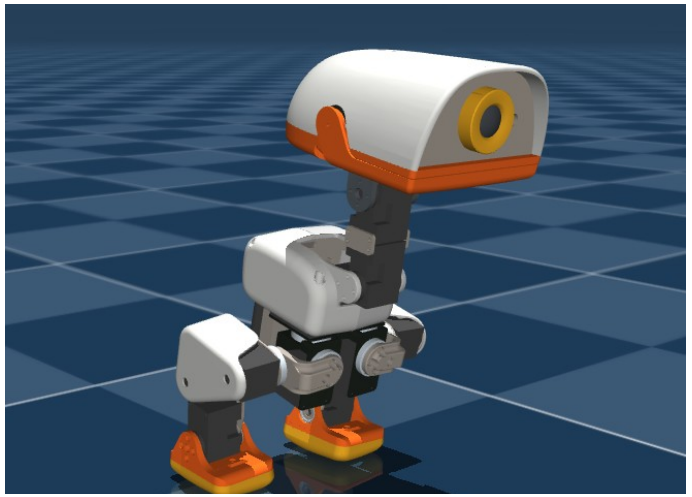
4. Y al robot real. El cerebro aprendido se copia tal cual.

Sin cerebro, se cae en menos de un segundo



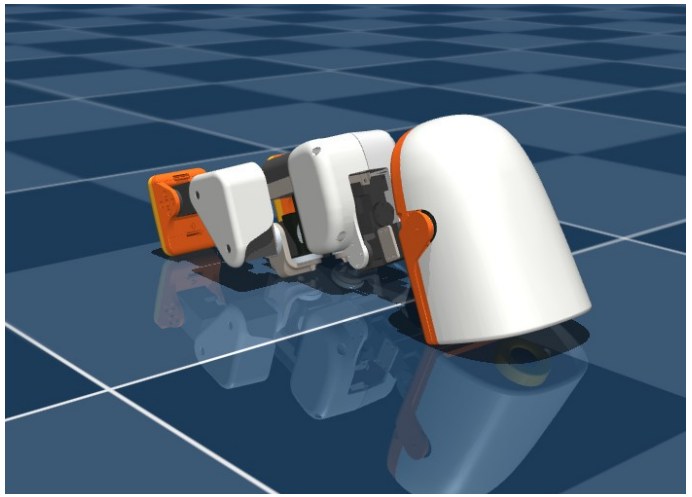
0 s

De pie



0,4 s

Se inclina



1 s

En el suelo

Aunque le pidamos «mantén la postura», sin IA no aguanta.

Cinco bloques, una idea cada uno

BLOQUE 1

Tu laboratorio

Lo montas y te mueves por él.

BLOQUE 2

El cuerpo

Del plano al gemelo digital.

BLOQUE 3

El cerebro

Cómo aprende una IA física.

BLOQUE 4

Laboratorio

Pon a prueba al robot.

BLOQUE 5

Realidad

Del gemelo digital al mundo.

Un laboratorio de IA física en tu PC

1

Python

El lenguaje de todo el taller. Exactamente la 3.12.

2

Una caja aislada

Un entorno virtual con sus propias librerías.

3

El simulador

MuJoCo calcula la física del robot.

4

El visor

La ventana 3D donde lo ves moverse.

No hace falta una GPU: todo corre en la CPU de un PC normal.

Lo que hiciste al empezar: Git y tu entorno

Git

Guarda la historia de un proyecto: cada cambio, quién y cuándo.

GitHub

La web donde vive el proyecto, a la que todos pueden acceder.

git clone

Copia el proyecto entero a tu PC. Es lo único que haces tú.

INSTALAR.bat creó una caja aislada (.venv) con las librerías de requirements.txt.

El mapa de tu laboratorio

Carpeta o fichero	Qué es	¿Lo tocas?
robot/	El cuerpo (robot_groundcontact.xml) y el mundo (scene.xml)	Solo scene.xml
policies/	Los cerebros: ficheros .onnx	No: se copian
exercises/	Los ejercicios: n1 mirar, n2 cambiar valores, n3 laboratorio	Sí: las constantes de arriba
scripts/	El programa que une el cuerpo con el cerebro	No
entrenar/	Cómo entrenar en casa: scripts y curvas reales	Se lee
TALLER.md · MISIONES.md	Tu hoja de ruta y las misiones	Se sigue

Cuerpo, mundo, cerebro y ejercicios: cuatro cosas que se cambian por separado.

Así se trabaja en cada ejercicio

1

Abre

El ejercicio en VS Code. Arriba hay unas constantes en mayúsculas.

2

Cambia UN valor

Solo uno, para saber qué lo causó. Antes, predice qué pasará.

3

Guarda y cierra

Ctrl+S, y cierra la ventana del robot: no se recarga sola.

4

Lanza y observa

Ejecútalo otra vez. ¿Acertaste?

Si no cambia nada: ¿guardaste? ¿cerraste la ventana?

TU TURNO · 30 MIN

Comprueba que tu laboratorio funciona

Un script revisa todo por ti: Python, librerías, el robot, las políticas y la simulación.

Debe terminar en: TODO LISTO

Y abrir el robot 5 segundos en el visor. Si es así, ya eres parte del laboratorio.

¿No arranca? Avisa, o trabaja con tu pareja.



El mapa de todo lo que verás hoy

Sensores
giro, inclinación, motores

➡

Observación
61 números

➡

Cerebro
red neuronal

➡

Acciones
14 órdenes

➡

Motores
XL330 con control propio

➡

Física
MuJoCo, 5 ms por paso

↺ se repite 50 veces por segundo: cada decisión del cerebro = 4 pasos de física de 5 ms

BLOQUE 2
Cuerpo y física
Motores y MuJoCo: cómo se mueve de verdad.

BLOQUE 3
Sentidos y cerebro
Las 61 entradas y la red de 197.896 números.

BLOQUE 4
Cómo se aprendió
De dónde salen esos números: el entrenamiento.

BLOQUE 5
Dónde falla
Cuándo el modelo deja de parecerse a la realidad.

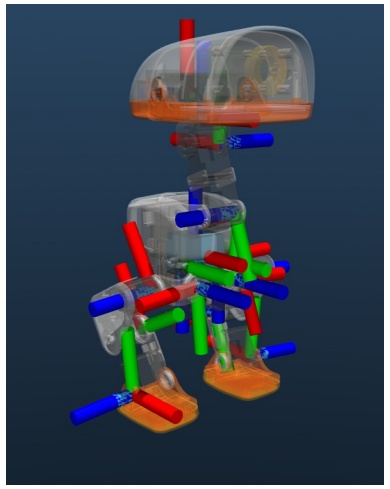
Cinco etapas para crear un gemelo digital



1

Modelo 3D

Las piezas.



2

URDF

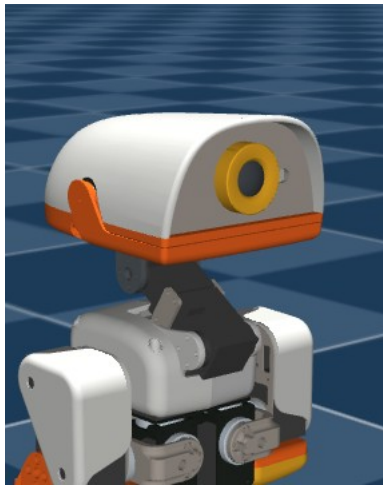
El esqueleto.



3

MJCF

La física.



4

Escena

El mundo.



5

Validación

Se sostiene.

El modelo 3D: dibujar las piezas

Un robot empieza como un dibujo en un programa de CAD. Cada pieza es una malla de miles de triángulos.

En el Microduck se diseña en **Onshape** y se exporta con la herramienta **onshape-to-robot**.

En la imagen, el mismo robot en «alambre»: cada línea es el borde de un triángulo.

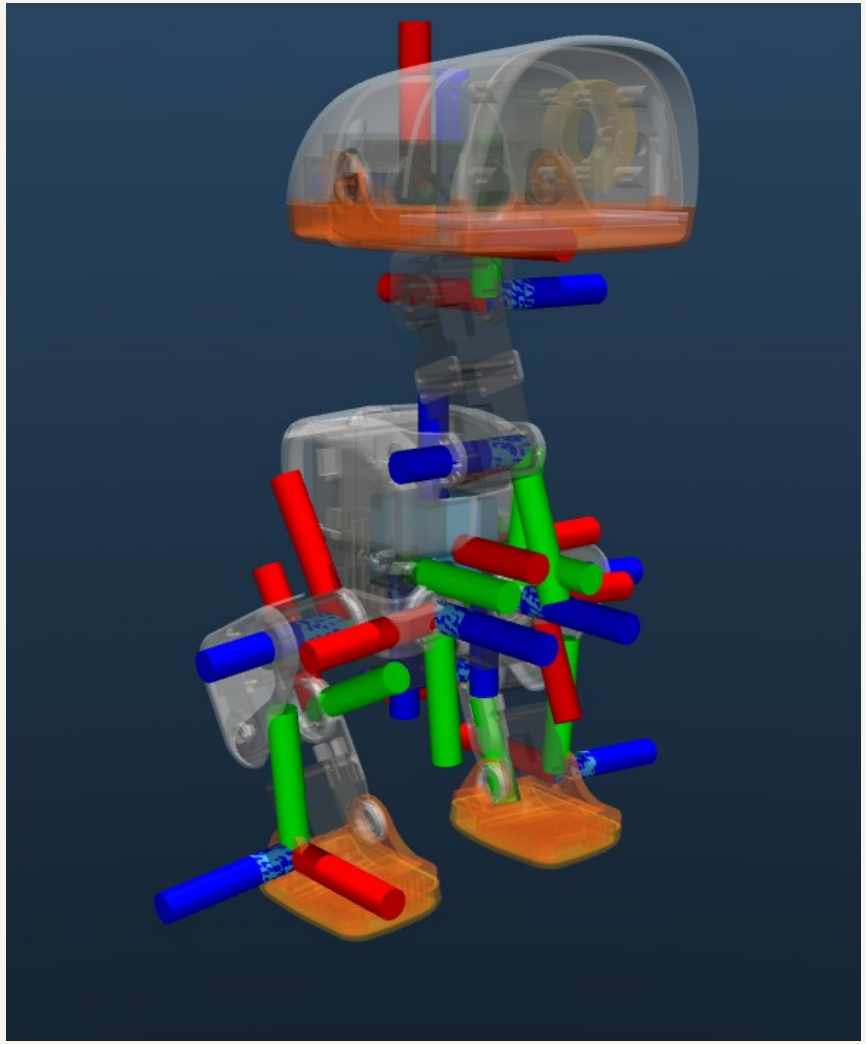


URDF: el esqueleto del robot

Un árbol de **piezas** (links) unidas por **articulaciones** (joints).

Los ejes de colores de la imagen son cada articulación: alrededor de ellos gira cada motor. El Microduck tiene 14.

Tronco → **Cadera** → **Rodilla** → **Tobillo**



El Microduck es un árbol de piezas

Tronco

Base libre: se mueve por el mundo

PIERNA IZQUIERDA

Cadera — 3 motores → **Rodilla** — 1 motor → **Tobillo** — 1 motor

5 motores

PIERNA DERECHA

Cadera — 3 motores → **Rodilla** — 1 motor → **Tobillo** — 1 motor

5 motores

CUELLO Y CABEZA

Cuello — 1 motor → **Cabeza** — 3 motores

4 motores

14 motores en total

14 acciones de la política

El Jacobiano: de los motores a los pies

ENTRADA

Mueves los motores

Cadera, rodilla y tobillo giran unos pocos grados.

LA TABLA

Jacobiano

Traduce unos movimientos en otros.

SALIDA

Se mueve el pie

Hacia dónde y con qué velocidad.

Responde a «si giro este motor, ¿cuánto se mueve el pie?» y, al revés, «¿qué fuerza hace el pie con cada motor?».

MuJoCo los calcula por dentro en cada paso. El aprendizaje por refuerzo no necesita que los escribas.

MJCF: añadir la física

El esqueleto no basta: el simulador necesita saber cuánto pesa cada pieza y cómo empuja cada motor.

Masas e inercias: las cajas rojas de la imagen.

Colisiones: qué piezas chocan con qué.

Actuadores: los motores, en negro.



URDF describe cómo está montado. MJCF añade cómo se comporta.

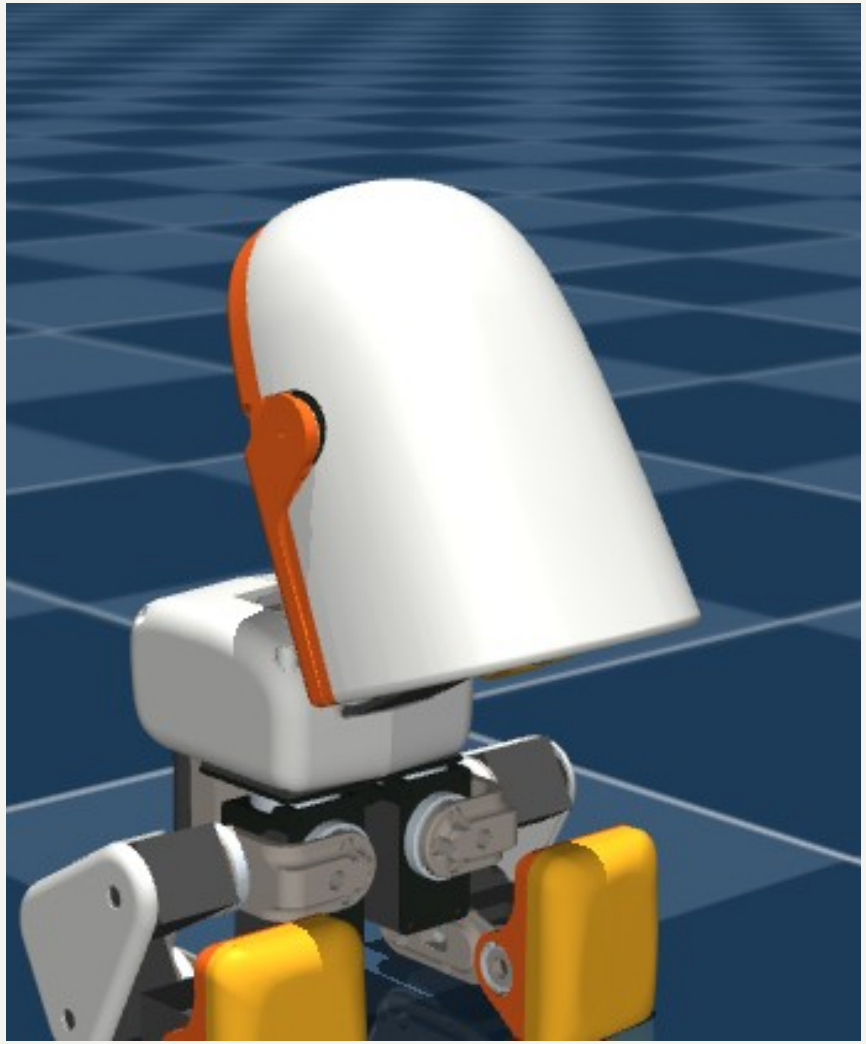
MuJoCo: el simulador de física

MuJoCo (Multi-Joint dynamics with Contact) es un simulador creado para robótica. Hoy es de código abierto.

Qué calcula: cómo se mueve cada pieza y cómo choca con el suelo y con las demás.

Cómo lo hace: muchísimos pasos por segundo, cada uno de una fracción de milisegundo.

En GPU: MuJoCo Warp, de NVIDIA, simula miles de robots a la vez.



Un paso de física: 5 milisegundos

Sensores
giro, inclinación, motores

→

Observación
61 números

→

Cerebro
red neuronal

→

Acciones
14 órdenes

→

Motores
XL330 con control propio

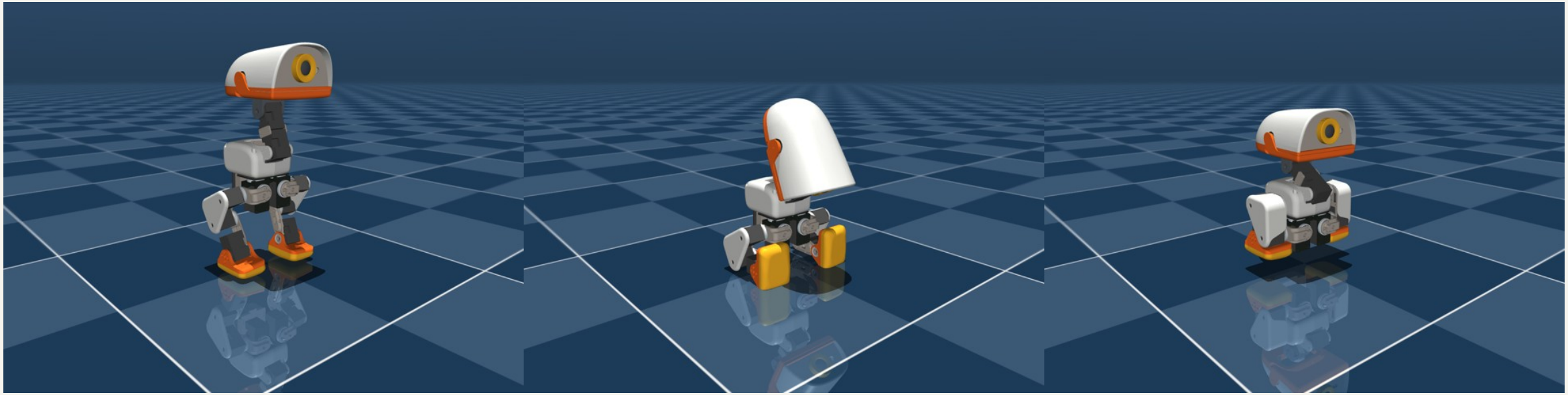
→

Física
MuJoCo, 5 ms por paso

↺ se repite 50 veces por segundo: cada decisión del cerebro = 4 pasos de física de 5 ms

Dato	Valor	Dónde se ve
Paso de simulación	0,005 s (200 por segundo)	timestep en infer_policy.py
Decisión del cerebro	cada 4 pasos = 20 ms (50 Hz)	decimation = 4
Qué manda el cerebro	ángulo objetivo = pose por defecto + acción × escala	metadatos del .onnx
El motor	Dynamixel XL330, modelo bam «m6»	BAM_MOTOR_NAME, BAM_MODEL
Límite de par	tensión × kt / R: baja con la batería	--vin, en bam

La escena: el mundo del robot



Suelo, luz y cámara. Todo lo que rodea al robot.

Posturas guardadas. De pie, sentado y plegado.

Objetos. Una versión con pelota para patear.

Validación: ¿se tiene en pie?

Lo abrimos en el visor y comprobamos que las piezas están bien puestas y que responde cuando lo empujamos.

Listo para ejecutar políticas.

Todavía sin cerebro: es solo el cuerpo.



TU TURNO

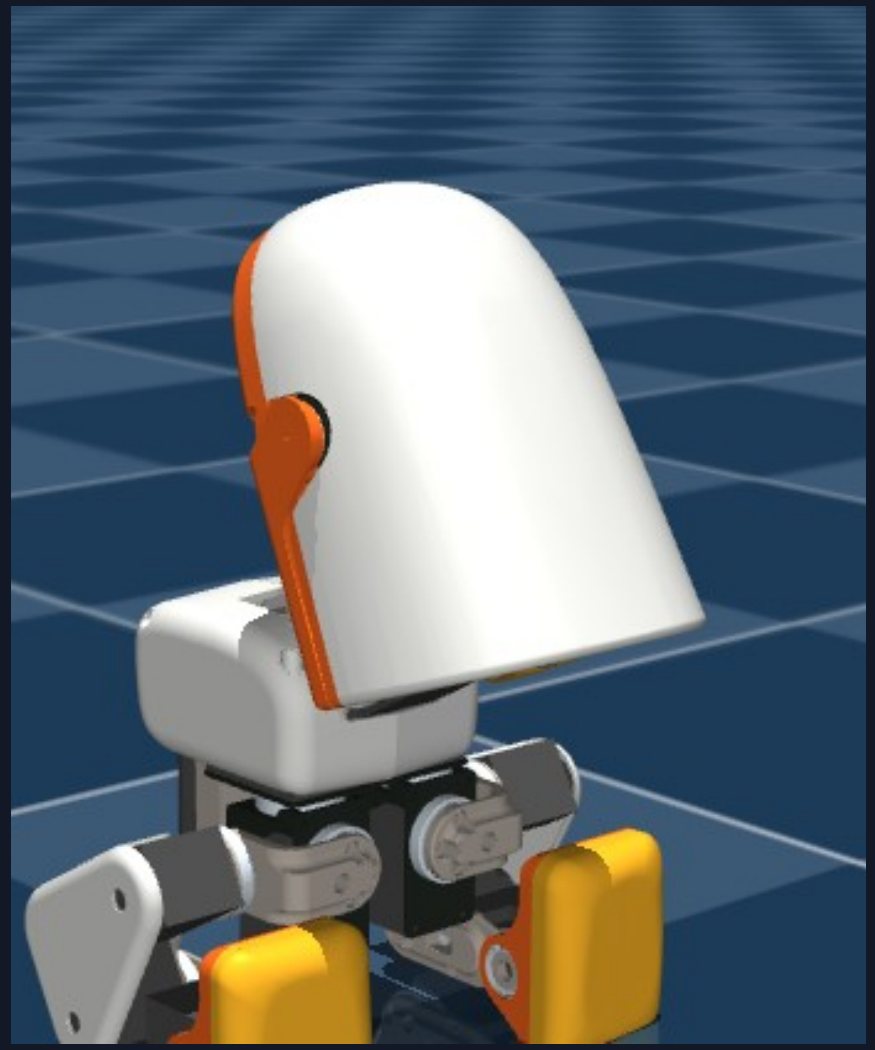
Cambia el mundo del robot

1. Abre el fichero de la escena y cambia los colores del suelo.

2. Guarda y vuelve a abrir el visor.

3. Cambia la postura: de pie, sentado o plegado.

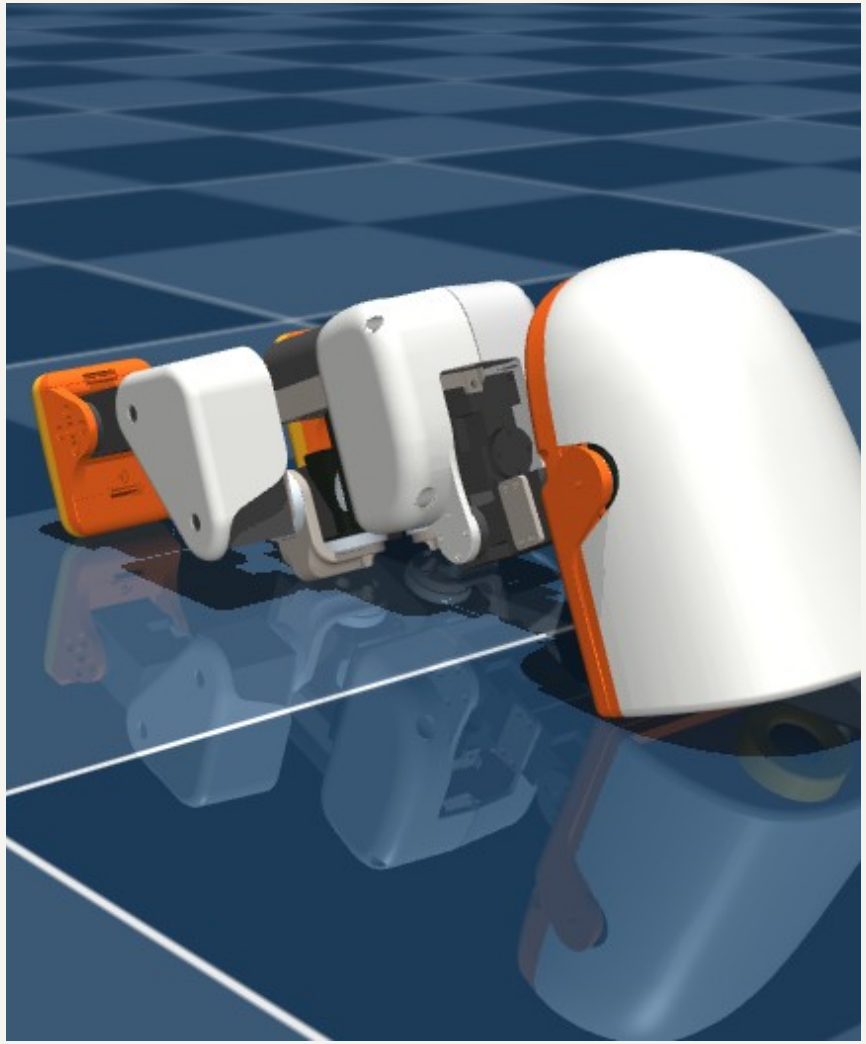
¿Qué cambia en el robot? ¿Y en el mundo?



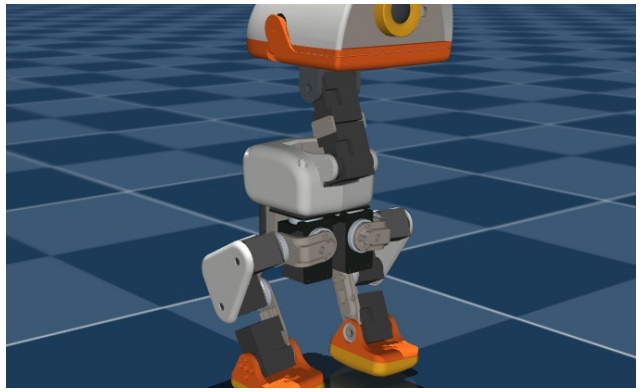
Aprender es como montar en bici

Nadie te da ecuaciones de equilibrio. Te subes, te caes, ajustas y lo intentas otra vez.

Con el robot pasa exactamente igual.



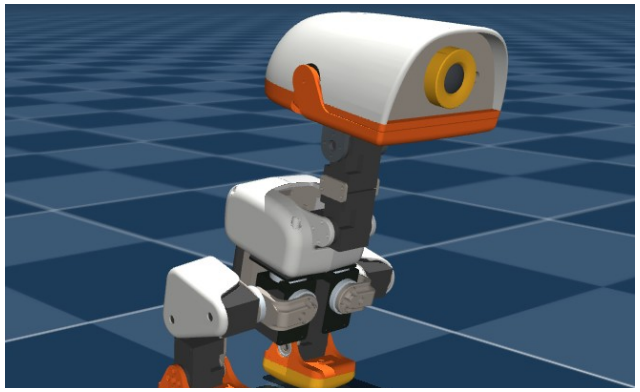
El bucle de aprender



1

Intentar

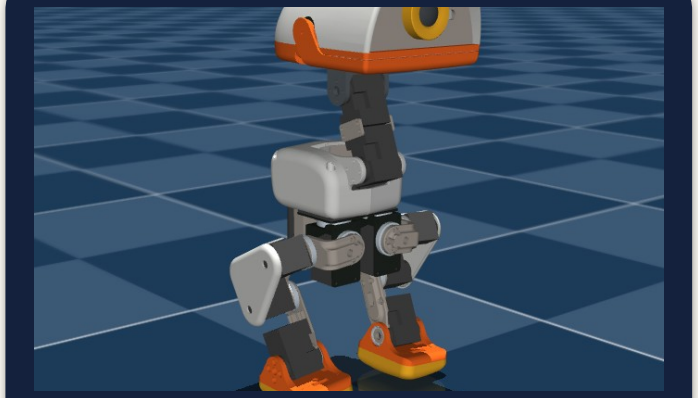
Mueve los motores.



2

Ver el error

Se inclina y cae.



3

Ajustar

Corrige un poco y repite.

Toda la IA de este taller es este bucle, repetido millones de veces.

Una red neuronal es una función con botones

Entran 61 números

Lo que siente el robot: inclinación, giro del cuerpo, posición de los motores y la orden de velocidad.

Miles de botones

Son los pesos de la red. Aprender es ajustarlos.

Salen 14 números

Los ángulos que debe tomar cada motor.

Cada 20 milisegundos: entran los sensores y salen los motores.

Dentro del cerebro: 197.896 números

Sensores
giro, inclinación, motores

→

Observación
61 números

→

Cerebro
red neuronal

→

Acciones
14 órdenes

→

Motores
XL330 con control propio

→

Física
MuJoCo, 5 ms por paso

↺ se repite 50 veces por segundo: cada decisión del cerebro = 4 pasos de física de 5 ms

La red

61 entradas → normalizar → 512 → 256 → 128 → 14 órdenes

Activación ELU. Son sumas y multiplicaciones: 197.896 pesos.

Las 61 entradas

3 giro · 3 inclinación · 14 posición

14 velocidad · 14 última orden

13 de orden (movimiento, cabeza, cuerpo)

Lo que lleva el fichero

Nombres de las 14 articulaciones, pose por defecto y los nombres de cada entrada: el .onnx se describe a sí mismo.

Cómo aprende: error y ajuste

1

Medir el error

El error (o loss) dice lo mal que va la red. Más bajo es mejor.

2

Seguir la brújula

El gradiente señala hacia dónde mover cada botón para que el error baje.

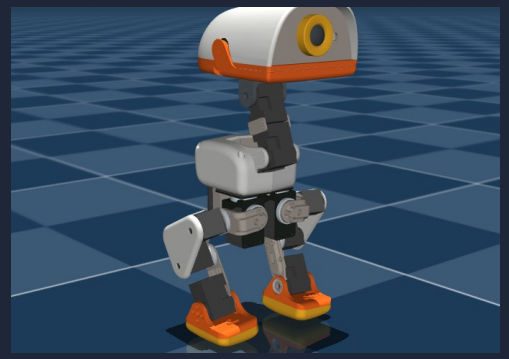
3

Dar un pasito

Se mueven los botones un poco y se repite. Una y otra vez.

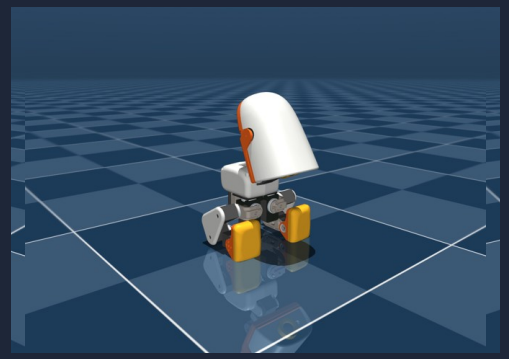
Es bajar una montaña con niebla: no ves el valle, pero notas hacia dónde baja el suelo.

Cuatro piezas para aprender sin profesor



Agente

Quien decide: la política del robot.



Entorno

El mundo donde actúa: el simulador MuJoCo.

ACCIÓN

14 ángulos

Uno por motor, 50 veces por segundo.

RECOMPENSA

Un número

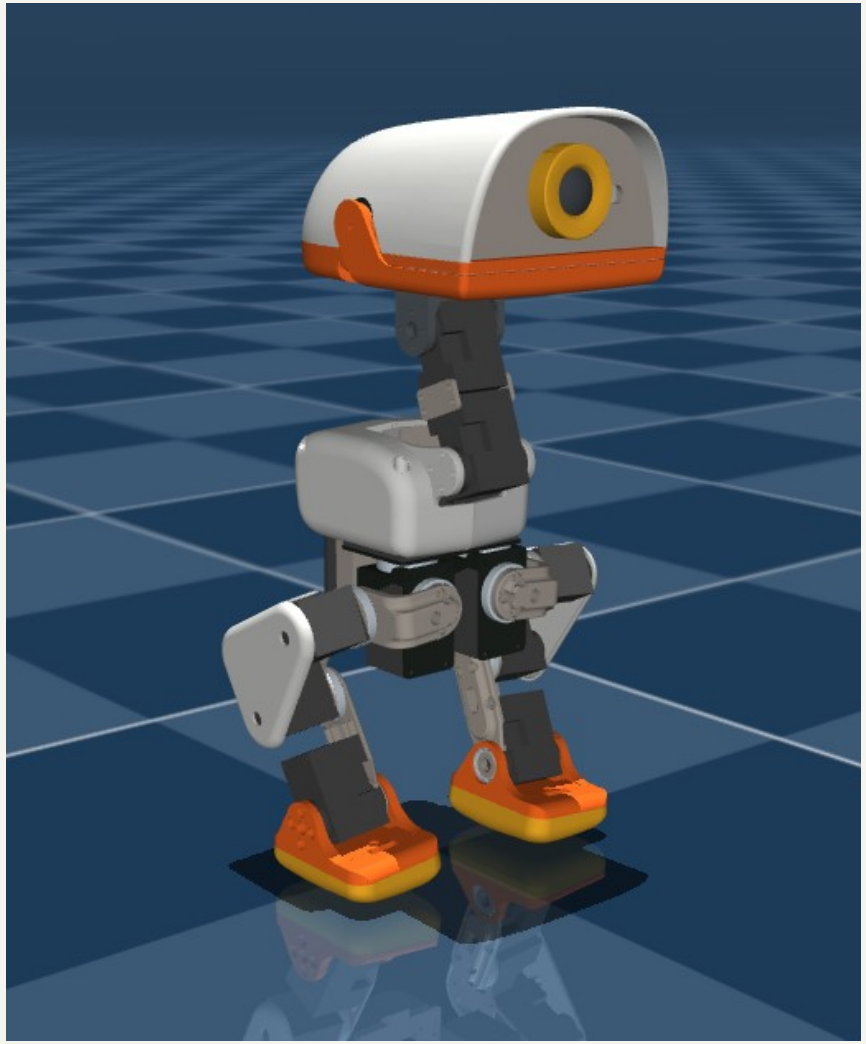
¿Lo hizo bien? Más alto, mejor.

¿Qué le damos como recompensa?

Premio por seguir la orden. Si le pedimos avanzar a cierta velocidad, gana puntos cuanto más se acerque.

Castigo por caerse. Si se cae, el intento termina y pierde lo que podía ganar.

Nadie dice cómo andar. Solo qué cuenta como hacerlo bien.



PPO: aprender con pasos pequeños

PPO (Proximal Policy Optimization) es el algoritmo con el que se entrena el Microduck. Es de los más usados en robótica.

1

Jugar

Miles de robots prueban la política actual.

2

Puntuar

Se ve qué acciones salieron mejor de lo esperado.

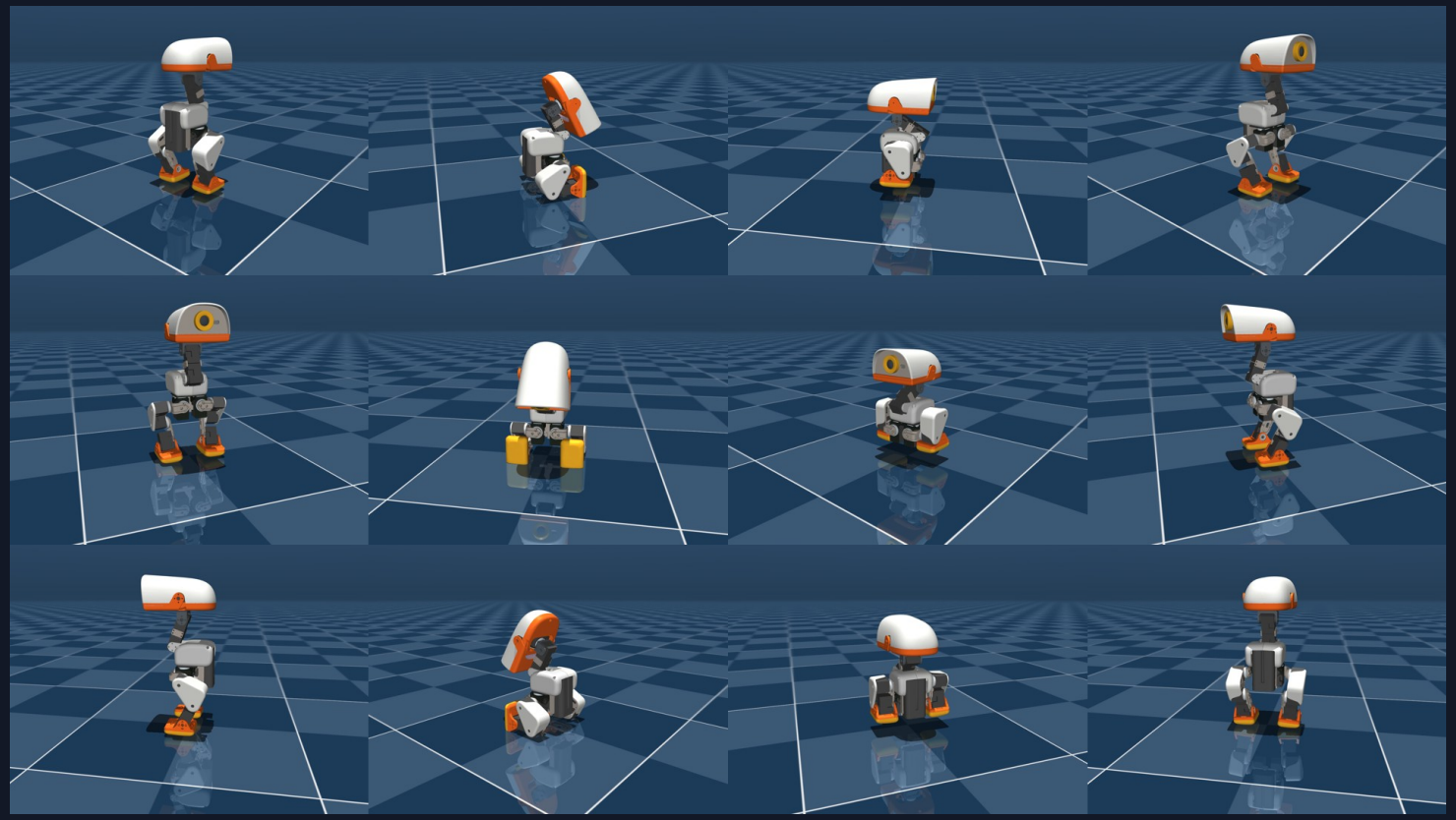
3

Mejorar un poco

Se refuerzan esas acciones, con un límite de cambio por paso.

Ese límite (en el Microduck, un 20 %) evita que un mal paso estropee lo ya aprendido.

Miles de patos aprenden a la vez



4096

robots simulados a la vez en una sola GPU NVIDIA.

Una hora o dos bastan para una marcha usable.

Cada recuadro de la imagen es un robot distinto.

Imitar o ganar puntos

Imitar a un humano

Es rápido: copia lo que ve. Pero nunca lo supera.

Aprender por recompensa

Puede superar al humano. Pero diseñar la recompensa es difícil.

La trampa: si solo premias «avanzar», puede aprender a arrastrarse o a lanzarse hacia delante.

TU TURNO · 25 MIN

Toca el cerebro

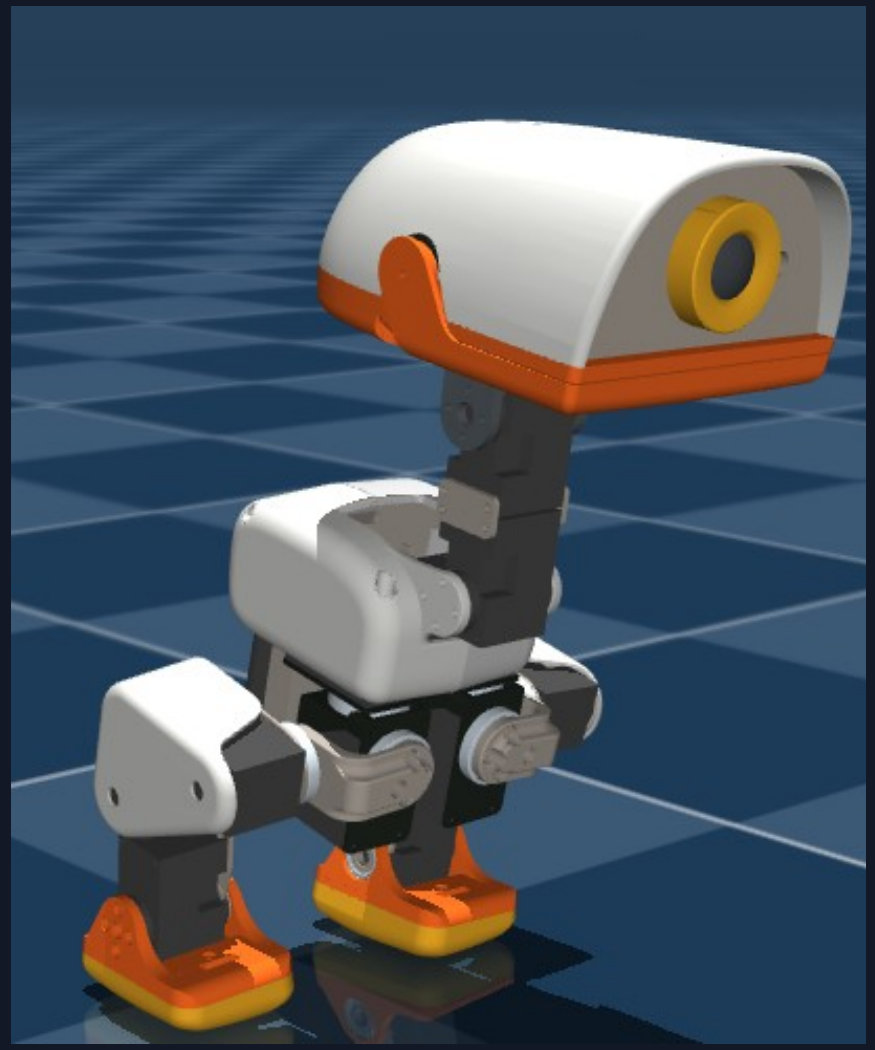
Nivel 1. En playground.tensorflow.org añade neuronas, cambia el learning rate y mira cómo la red separa los puntos.

Nivel 2. Mira dentro del cerebro del pato: qué números entran y qué órdenes salen. Quítale un sentido y mira si se cae.

Nivel 2. Mira la nota que recibe el pato en cada paso y cambia los pesos.

Nivel 2. Entra en la red: mira los datos pasar capa a capa y apaga neuronas.

Cambia UN valor cada vez y descubre de qué depende el pato.



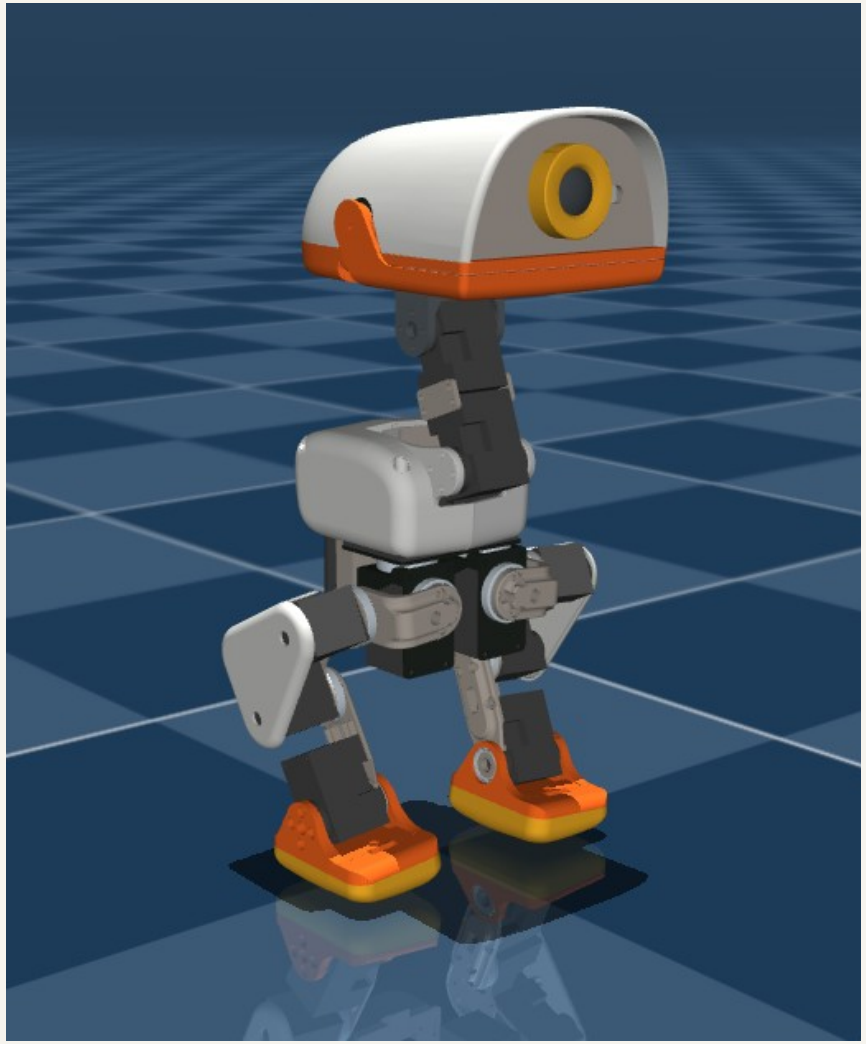
Una política es un fichero

Después de entrenar, todo lo aprendido se guarda en un archivo: el cerebro del robot.

Formato ONNX. Un estándar para guardar redes entrenadas.

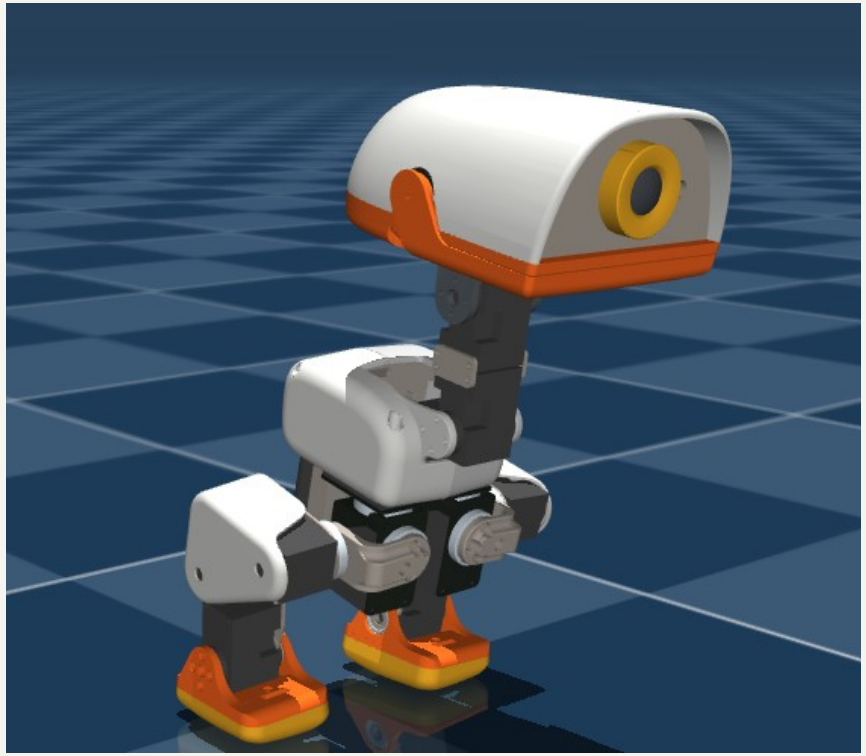
Se ejecuta en cualquier PC. No hace falta GPU para usarlo.

Siempre 61 entradas y 14 salidas. Por eso puedes cambiar de cerebro.



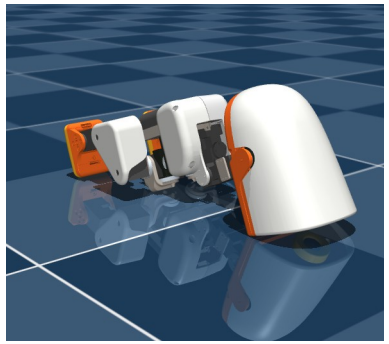
Pon a prueba al robot: cambia el mundo

Qué cambias	Qué simula	Pista
El suelo	Hielo o goma	friction 0.1, 1, 2
La batería	Voltaje que cae	6.5 voltios
El retraso	Motores que obedecen tarde	2 a 4 pasos
La potencia	Motores débiles	0.7
El cerebro	Otra política	otro fichero



Cambia una sola cosa cada vez y empuja al robot.

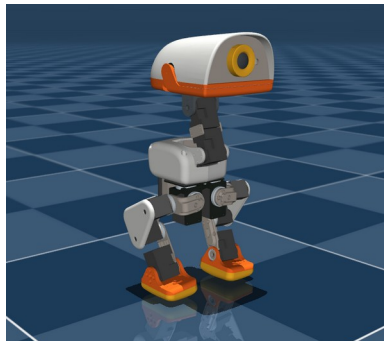
El mismo robot, más y más entrenado



ITERACIÓN 0

Recién nacido

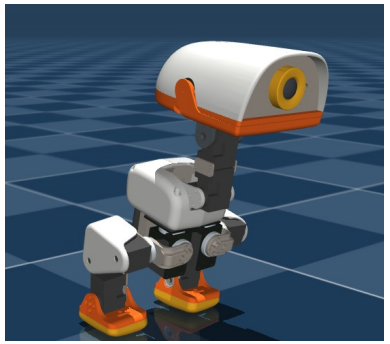
Se cae en un segundo.



ITERACIÓN 50

Quieto

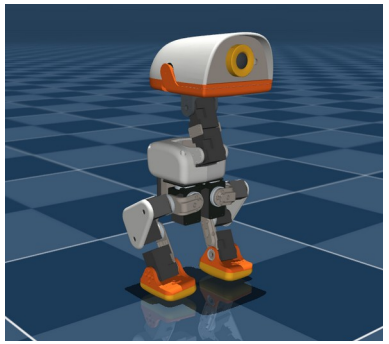
Se mantiene, pero no avanza.



ITERACIÓN 200

Tambaleante

Avanza, pero se desvía.



ITERACIÓN 400

Bien

Avanza con soltura.



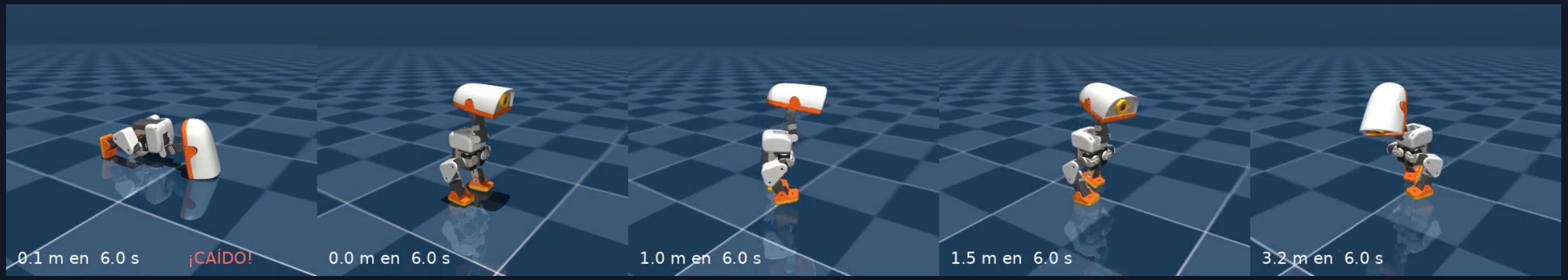
ITERACIÓN 1499

Mejor

Anda lejos y sin caerse.

Es el mismo entrenamiento guardado en distintos momentos. Menos intentos, peor cerebro.

La carrera de los cerebros



Misma orden para los cinco: avanzar. Solo cambia lo entrenado que está cada cerebro.

Simulación propia en MuJoCo · cerebros entrenados por nosotros con microduck_rl

Del entrenamiento al robot real

1

Entrenar

Miles de robots simulados en una GPU.

2

Evaluar

Mirarlo andar en el visor.

3

Exportar

Guardarlo como fichero ONNX.

4

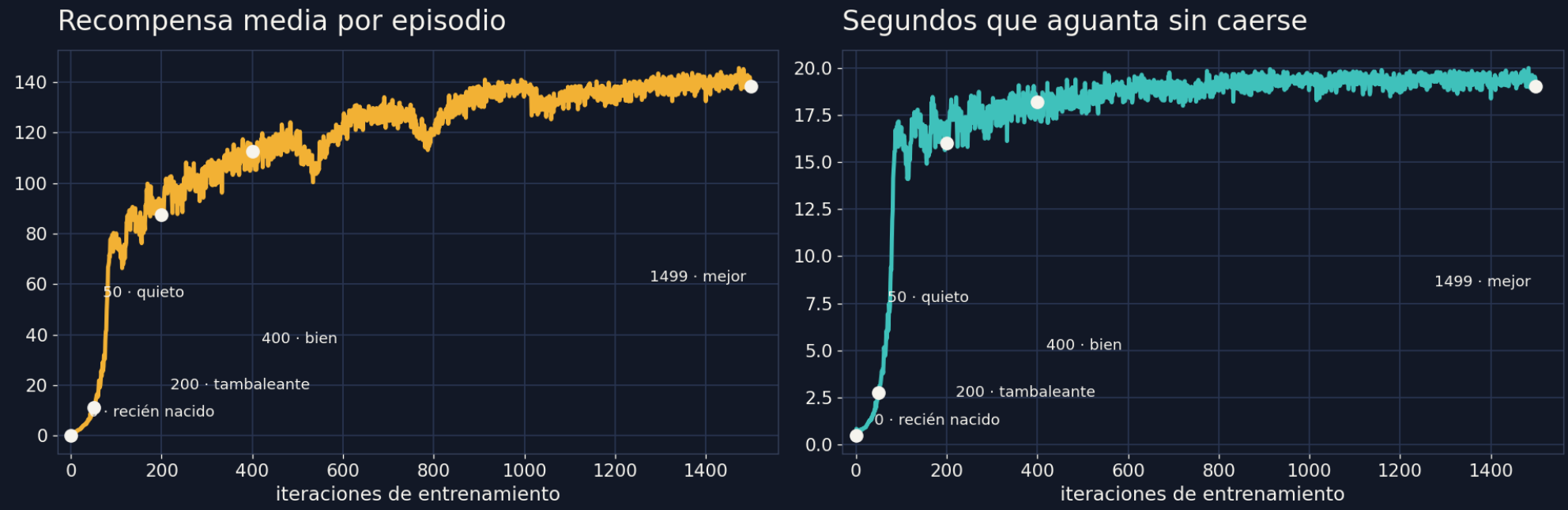
Desplegar

Cargarlo en el robot real.

Hoy hacemos el paso 2 con políticas ya entrenadas. Los pasos 1 y 3 se hacen en casa, con GPU.

Así se entrena de verdad

Entrenar al Microduck: 47 minutos en una GPU, 4096 robots a la vez




```
uv run train Mjlab-Velocity-Flat-MicroDuck --env.scene.num-envs 4096 --agent.max-iterations 1500
```

En casa, con una GPU NVIDIA. Hoy usamos los cerebros que ya salieron de este entrenamiento.

El ciclo del entrenamiento

1

Tarea

Qué debe aprender y qué se premia.

2

Jugar

4096 robots prueban el cerebro actual.

3

Puntuar

Cada intento recibe una recompensa.

4

Ajustar

PPO mueve los botones hacia lo que dio más puntos.

5

Repetir

Cientos de rondas. Se guarda una copia cada 25.

Toda la IA de este taller es este bucle, repetido millones de veces.

Los botones del entrenamiento

Botón	Valor	Qué controla
num-envs	4096	Robots a la vez. Más robots, más experiencia por ronda.
learning-rate	0,001	Tamaño de cada ajuste. Muy alto se descontrola; muy bajo casi no avanza.
entropy-coef	0,01	Premio por probar cosas distintas (exploración).
gamma	0,99	Cuánto importa el futuro frente a lo inmediato.
clip	0,2	Límite de cambio en cada ajuste: protege lo ya aprendido.
max-iterations	300	Cuántas rondas de entrenamiento.

Cada botón es una opción del comando de entrenamiento.

Una ronda de entrenamiento por dentro

1

Jugar

4096 robots × 24 pasos = 98.304 experiencias.

2

Puntuar

Cada paso recibe su recompensa; la ventaja dice cuánto mejor fue de lo esperado (γ 0,99 · λ 0,95).

3

Ajustar

5 épocas × 4 minilotes. El recorte 0,2 limita cuánto cambia el cerebro de golpe.

4

Vigilar

La divergencia KL adapta el aprendizaje; la entropía 0,01 mantiene la exploración.

Esto se repite 300 veces: casi 30 millones de experiencias para aprender a levantarse.

Así lo harás tú en casa

1

Preparar

Instala todo y comprueba que la GPU funciona.

2

Entrenar

4096 robots aprenden a la vez, unos 10 minutos.

3

Mirar

Si la recompensa sube, el robot está aprendiendo.

```
uv run train Mjlab-StandUp-Flat-MicroDuck --env.scene.num-envs 4096 --agent.max-iterations 300
```

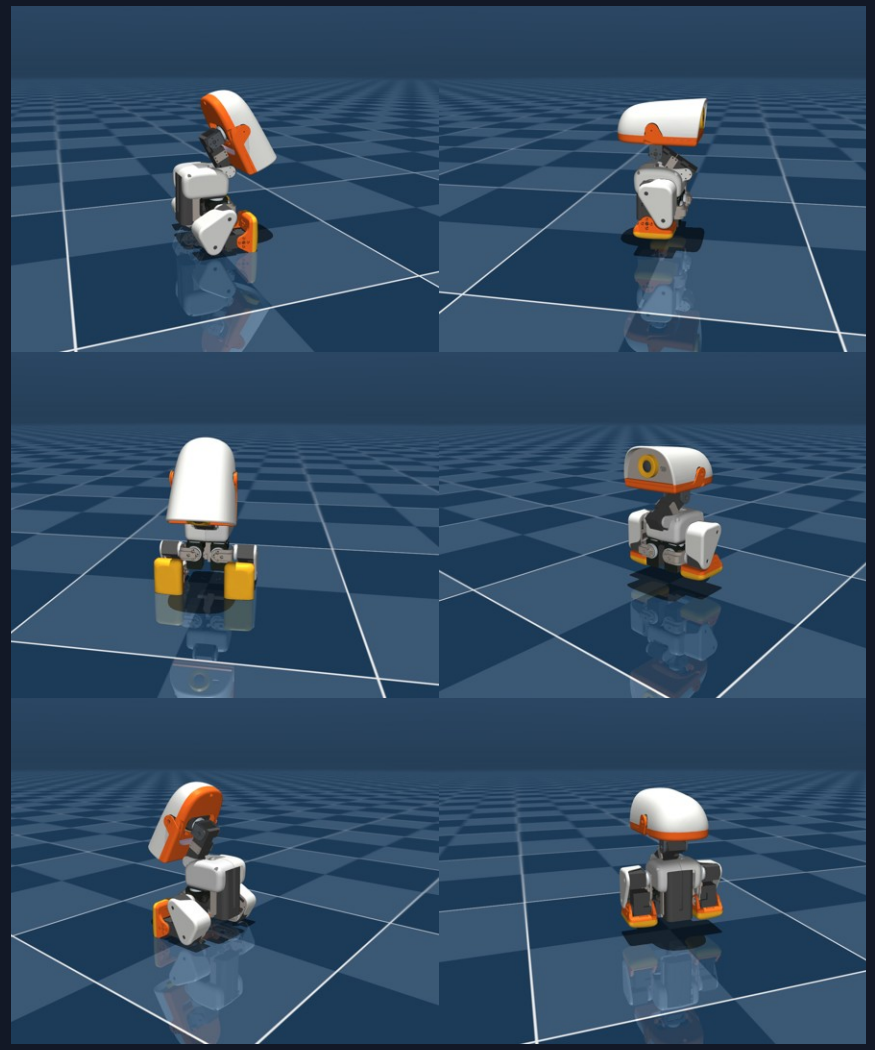
Todo explicado en la carpeta entrenar/ del repositorio. Hace falta una GPU NVIDIA.

TU TURNO · 15 MIN

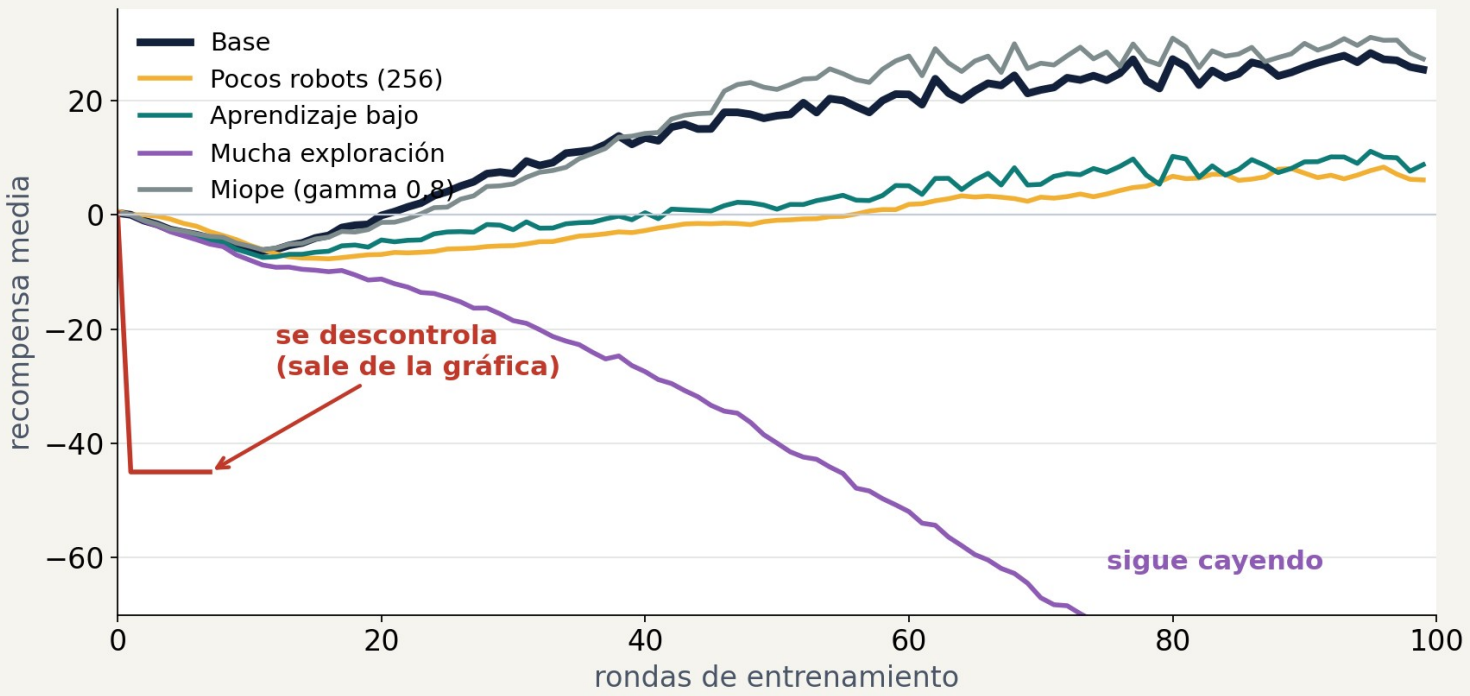
Predice y comprueba

1. Abre exercises/n3_04_curvas_entrenamiento.py y F5.
2. Lee qué se cambió en cada experimento.
3. **Antes de marcar la casilla, apunta tu predicción** en MISIONES.md (misiones 12 a 15).
4. Marca la casilla: ¿acertaste?

Son curvas reales: mismo robot, mismos 100 intentos, un solo botón cambiado cada vez.



Lo que pasó de verdad



Pasos grandes: se descontrola.

Mucha exploración: nunca se decide.

Pasos pequeños o pocos robots: aprende, pero despacio.

Mirar lejos: aquí no hacía falta; el premio llega en cada paso.

Tarea: levantarse del suelo · 100 rondas · mismo punto de partida · RTX 4090

La configuración de un entrenamiento

```
# tarea
Mjlab-StandUp-Flat-MicroDuck
# entorno  (env.yaml)
num_envs: 4096          # robots a la vez
decimation: 4 × 0,005 s  # = 50 Hz
episode_length_s: 6
# algoritmo PPO  (agent.yaml)
learning_rate: 0.001  (adaptive)
gamma: 0.99    lam: 0.95
clip_param: 0.2    entropy_coef: 0.01
num_learning_epochs: 5    num_mini_batches: 4
num_steps_per_env: 24
# red  (actor y critic)
hidden_dims: [512, 256, 128]   activation: elu
```

Dos ficheros, todo el entrenamiento

Cada carrera guarda su configuración en params/env.yaml y params/agent.yaml.

Cualquier valor se cambia al lanzar:

--agent.algorithm.learning-rate 0.0005

--env.scene.num-envs 2048

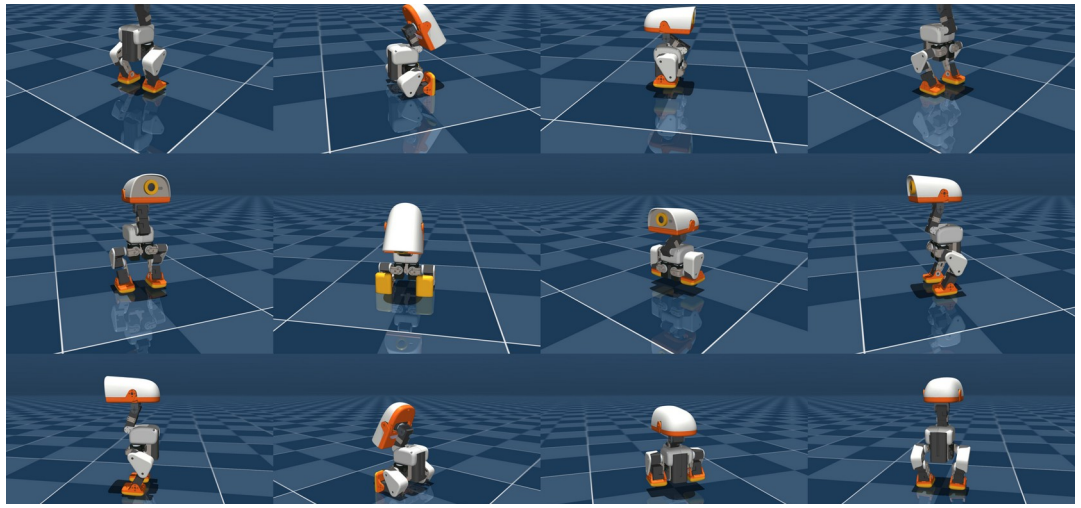
--agent.max-iterations 500

La recompensa: una suma con pesos

Pieza de la recompensa	Peso	Qué premia o castiga
height_stand_sharp	+1,0	Tener el tronco a la altura de pie
upright_sharp	+1,5	Estar bien vertical (margen de unos 6°)
standing_composite	+3,75	Puntuación combinada de estar de pie
pose_stand_legs	+2,0	Piernas en la postura de pie
action_rate_l2	−0,1	Castiga sacudir los motores
dof_pos_limits	−1,0	Castiga forzar las articulaciones al límite
self_collisions	−1,0	Castiga chocar consigo mismo

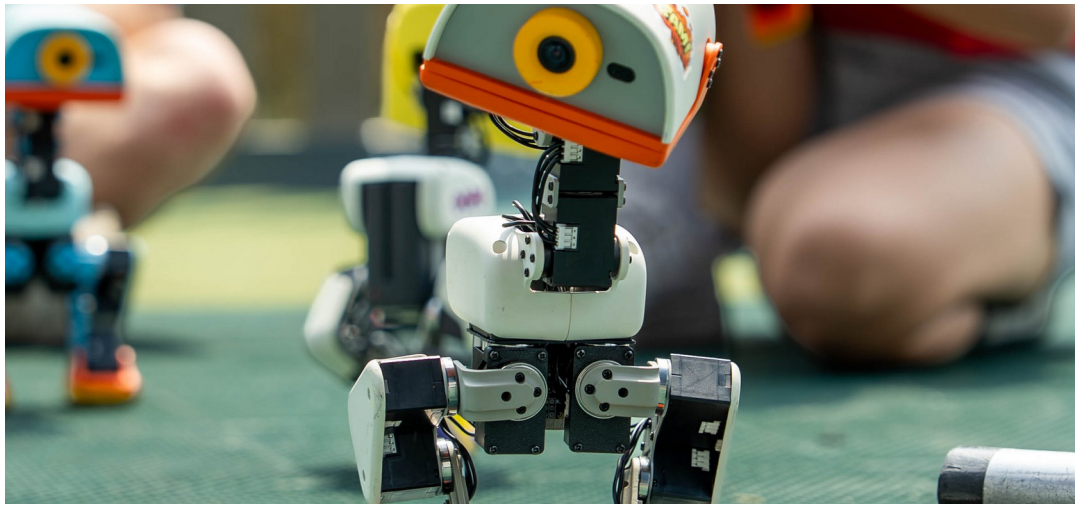
Cambiar un peso cambia lo que aprende: --env.rewards.upright-sharp.weight 0

La brecha entre simulación y realidad



Simulación

Física perfecta y repetible.



Realidad

Holguras, batería, retrasos, suelos distintos.

Tres culpables en los motores

Juego mecánico (backlash). Los engranajes de un servo pequeño tienen holgura.

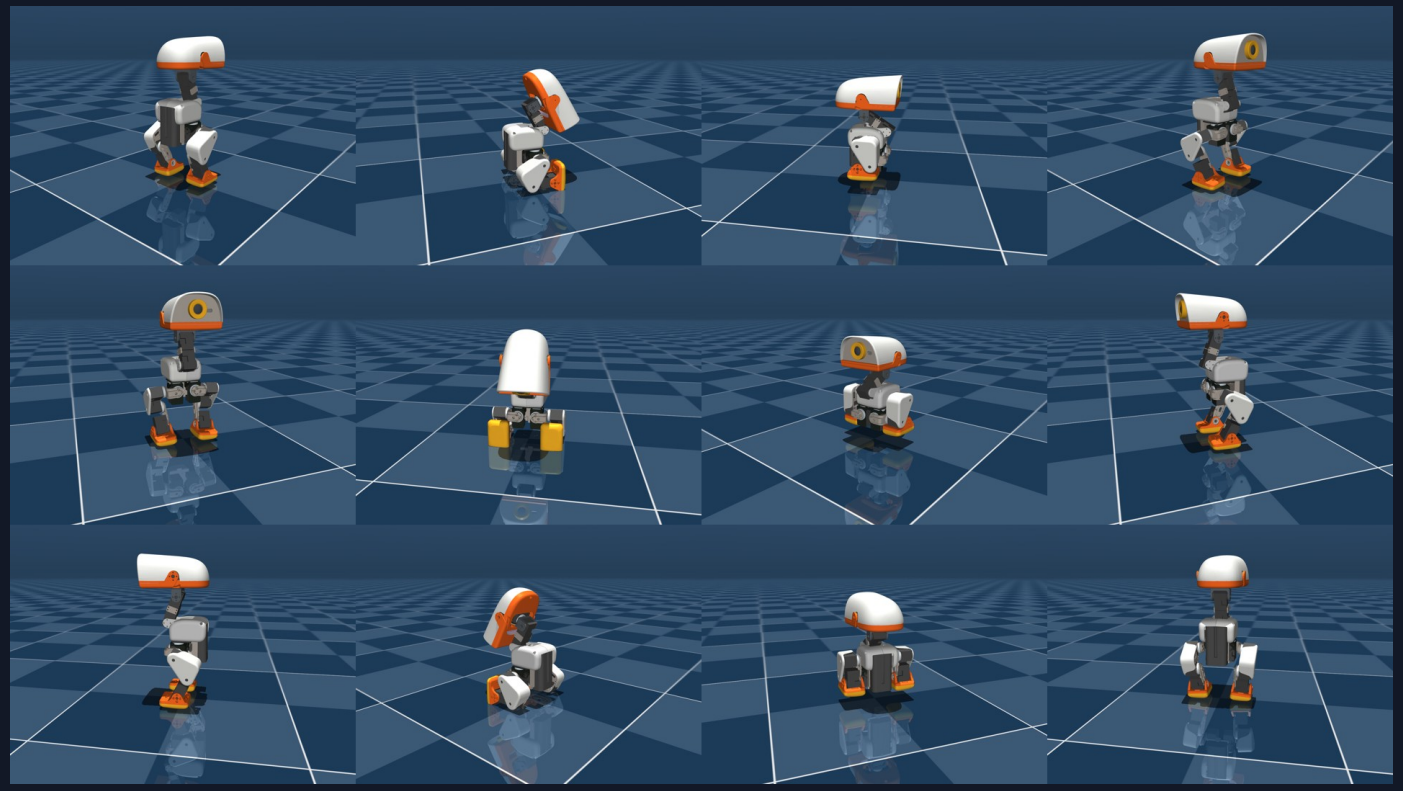
Voltaje de batería. Si la batería cae, el motor empuja menos.

Retraso de comunicación. La orden llega unos milisegundos tarde.

La solución: un modelo de motor que incluya estos efectos.



Entrenar con el mundo al azar



En cada intento se cambia al azar:

· masa y centro de masas

· fricción del suelo

· inercia de las articulaciones

· voltaje de la batería

· retraso de las órdenes

Así no depende de un valor exacto.

Lo que ya hace el robot real



Se recupera de un empujón



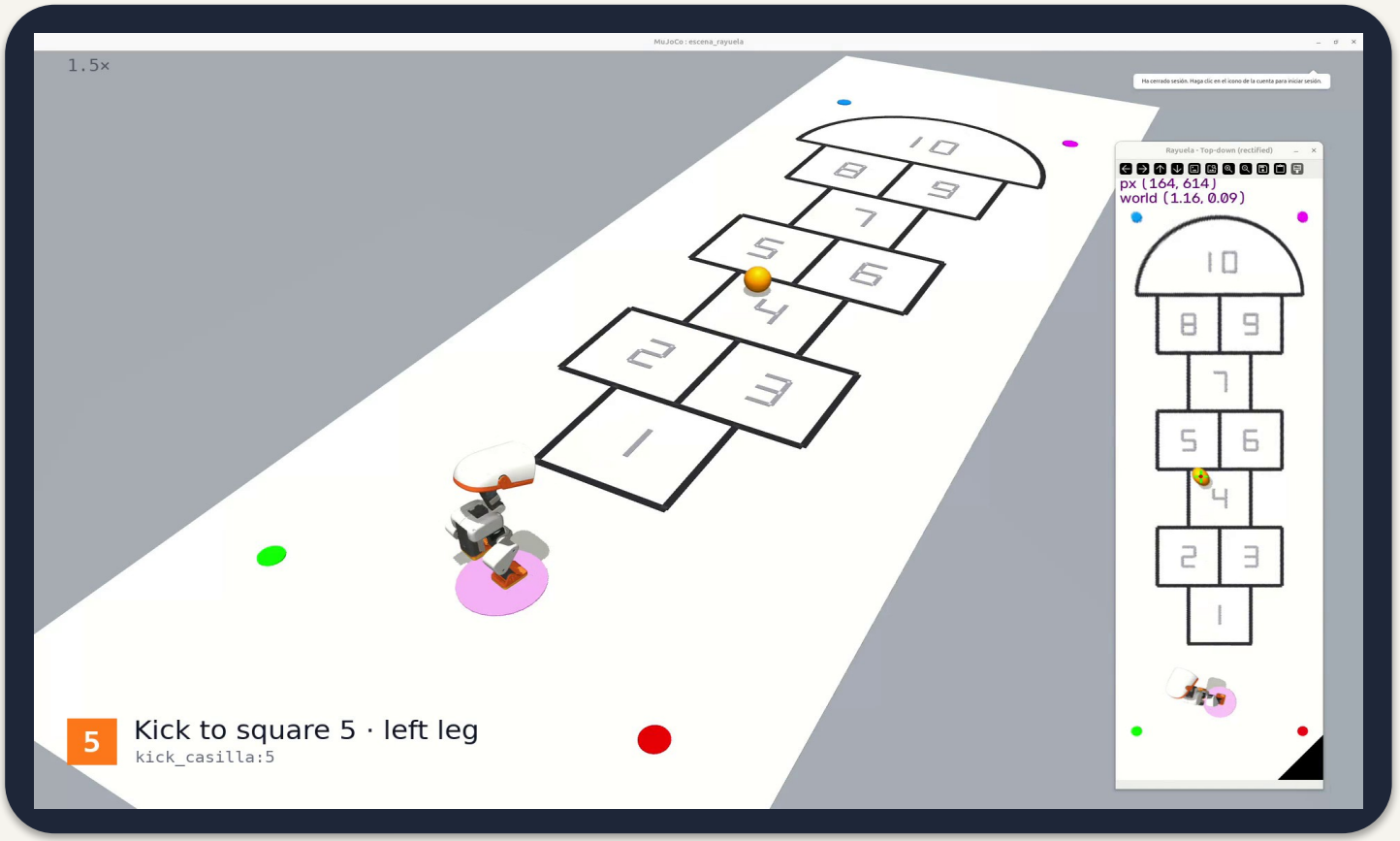
Un coro de patos



Agarra y lleva

Vídeos del Microduck real: Pollen Robotics (press kit oficial).

Rayuela: el Microduck juega



Simulación en MuJoCo + ROS 2 · Bravo Robots

Tres pasos, tres IA

1. Patea. Una política entrenada para golpear la pelota con la fuerza justa.

2. Mira. Una cámara calcula en qué casilla ha caído.

3. Va. Otras políticas lo llevan andando o rodando hasta allí.

github.com/Bravo-Robots/microduck_rl-rayuela

CIERRE

Cinco ideas para llevarte

1

Aprender

Error y ajuste, repetido muchas veces.

2

Refuerzo

Probar y recibir una recompensa.

3

Gemelo digital

Un cuerpo virtual donde equivocarse gratis.

4

Política

El cerebro: un fichero que se puede copiar.

5

Sim-to-real

El reto: que funcione fuera.

Entrenar el tuyo necesita una GPU NVIDIA, MuJoCo y el algoritmo PPO. Ya sabes qué hace cada uno.

PRÓXIMA MISIÓN

Gracias

Reentrena el cerebro con una GPU NVIDIA.

Cambia la recompensa y mira qué aprende.

Prueba otro robot o MuJoCo Playground.



Recursos: pollen-robotics/microduck_rl · pollen-robotics/microduck-policies (Hugging Face) · tutorial de NVIDIA «Microduck con Isaac Lab» · MuJoCo y mjlab

Robot y mallas 3D: Pollen Robotics (CC BY-NC-SA 4.0). Fotos y vídeos reales: Pollen Robotics. Imágenes del simulador: renders propios.

Upskilling Week
Barcelona

Thanks for coming!

Please leave your feedback here.

It will take no more than 5 minutes

Impulsat per

Mobile WorldCapital Barcelona

Amb la col·laboració de

Barcelona | Barcelona Activa | Generalitat de Catalunya

Powered by

TALENT ARENA